Compiler Optimizations for Attention and Functional KV Caches

CS6886: Systems Engineering for Deep Learning 1 November 2026

Nishanth Senthil Kumar (EE23B049) IIT Madras

1 Why the two IREE flags remove the original GQA repeat

Starting graph. GQA shares H_{kv} key/value heads among H_q query heads [1, 2]. For 8 query heads and 2 KV heads, $g = 4$: query heads 0–3 use KV head 0 and heads 4–7 use KV head 1. With batch omitted, $Q : [8, M, D]$ and $K, V : [2, N, D]$:

```
# Natural decomposed attention: expanded tensors have 8 heads.
Kr = repeat_interleave(K, 4, dim=0)   # [8, N, D]
Vr = repeat_interleave(V, 4, dim=0)
P  = softmax((Q @ Kr.transpose(-2, -1)) / sqrt(D), dim=-1)
O  = P @ Vr
```

The saved Linalg IR first broadcasts K/V to $[2, 4, N, D]$, collapses the two head axes to 8, then feeds named `linalg.batch_matmul` operations. The broadcast is a tensor-producing operation; reshaping alone is not evidence of a copy.

Flag 1: expose matmul as a generic contraction

`--iree-opt-generalize-matmul`

The `GeneralizeLinalgNamedOps` pass rewrites named matmuls as `linalg.generic`, exposing parallel/reduction loops, operand indexing maps and multiply-add arithmetic [13, 14]. A reduction sums many feature products into one output:

```
# Schematic loop meaning of linalg.batch_matmul(Q, Kr.T):
for h, m, n:                    # parallel output indices
    S[h,m,n] = sum_d(Q[h,m,d] * Kr[h,n,d])
```

Why needed: the reshape pass has rules for generic operations; the pinned implementation excludes propagation through named Linalg operations. Generalization exposes the structure those rules need. It does not itself guarantee fusion or a faster kernel.

Flag 2: propagate reshapes through reductions

`--iree-dispatch-creation-enable-aggressive-reshape-movement`

The reduction-crossing option in `BubbleUpExpandShapes` permits expand/collapse propagation through eligible generic reductions, rather than restricting it to all-parallel operations [13, 14].

Effect here: the flattened head axis can be split into *KV head×sharing group*. Existing elementwise fusion can then combine the broadcast with the contraction and compose its indexing map. The resulting loop meaning is:

```
for hk in range(2):
  for j in range(4):
    h = 4*hk + j
    S[h,m,n] = sum_d(Q[h,m,d] * K[hk,n,d])  # no Kr
# The probability-times-V contraction analogously reads V[hk].
```

This equals $h_{kv} = \lfloor h_q/4 \rfloor$, but the compiler can retain separate `hk,j` loops without emitting an explicit integer divide. This changes loop shapes/indexing while preserving the complete dot product. The documentation is therefore under general Linalg/shape-fusion passes, rather than a GQA-specific rule.

Evidence and implication for our original pass

Generalize matmul	Aggressive reshape	Broadcast calls	All dispatch calls
Off	Off	2	5
On	Off	2	5
Off	On	2	5
On	On	0	3

Fresh O3 CPU compile-to-Flow check: $H_q = 8, H_{kv} = 2$. Observed order: generalization, reshape movement, then elementwise fusion. Final contractions consume compact $[2, N, D]$ K/V. Earlier ablation: 20 fixtures, 160 compilations and 96 numerical checks.

What becomes redundant? Our pass manually constructs grouped contractions; these flags plus fusion already eliminate the tested repeats. The original 1.86× default CPU gain therefore does not show an additional capability over strong IREE. O3 alone is insufficient. This is not all-graph coverage: MQA can need fewer flags, and extra or escaping users can retain expanded tensors.

2 Functional-cache transformations: what changes and why

Examples are explanatory pseudocode. `update(C,p,U)` returns a functional tensor value with U replacing rows $p : p + t$; it does not mean every backend must allocate/copy the whole C. Let $C : [P, H, D]$ and $U : [t, H, D]$.

2.1 Conditional read forwarding: avoid building an unused full version

Before: each candidate forms an updated cache, but downstream sparse attention only gathers a few rows.

```
T = update(C, p, U)             # logical full updated version
R = gather(T, ids)
# After: conditional loads from original history or the patch.
R = stack([U[i-p] if p <= i < p+t else C[i] for i in ids])
```

Example: $P = 10000, p = 9000, t = 1$, `ids=[0,9000,9999]` returns `[C[0],U[0],C[9999]]`. If T has only supported read consumers, its full materialization can disappear. **Use case:** several branches share a large pool but each reads a small subset. **Limits:** required full-cache outputs remain; supported types, indices, slices and uses are checked. Read-over-write is classical array algebra [3, 4], not a new identity.

2.2 Masked gather forwarding: a different implementation of the same law

Instead of per-element conditional control flow, gather a compact base and patch and select their elements. The implemented masked variant uses a safe patch index even outside the update:

```
inside = (p <= ids) & (ids < p+t)
safe   = where(inside, ids-p, 0)  # U must have at least 1 row
base   = gather(C, ids)
patch  = gather(U, safe)
R      = where(expand_mask(inside), patch, base)
```

Why the safe index matters: a select may evaluate both operands; an out-of-bounds patch read cannot be justified by a later mask. **Tradeoff:** masks, two gathers and selection add work. A demand/capacity cost gate rejects dense cases; the masked form also has an irregular-shape CUDA compile failure with a conditional fallback. It is not universally faster.

2.3 Winner-cache factoring: select the small update before commit

Two candidates share the same history but propose different new rows. The final choice is a scalar Boolean:

```
# Before: selection consumes two full appended cache values.
Ta, Tb = concat(C, a), concat(C, b)
committed = select(winner_is_a, Ta, Tb)
# After: move the same selection to the small appended rows.
committed = concat(C, select(winner_is_a, a, b))
```

Example: with 4096 cached rows and one new row, selection moves from 4097-row tensors to one-row tensors. The pass also supports equal-region insert-slice forms. It checks a common base, axis/region, types and scalar condition; different histories or elementwise conditions reject. **Use case:** branching inference commit. The pretrained driver has 60 concat matches; ordinary decode has zero. Both candidate attention computations still need their inputs, so this does not prove all candidate caches or copies disappear.

2.4 Adjacent-update coalescing: build one small patch

```
# Before: intermediate versions are not observed elsewhere.
C1 = update(C, 100, a)           # a has 1 row
C2 = update(C1, 101, b)          # b has 1 row
# After:
C2 = update(C, 100, concat(a, b))
```

Use case: supported multi-token update chains before forwarding. Adjacent regions become one small patch; fewer intermediate updates can expose later elimination. Guards require compatible slices, positive sizes, unit strides, no gap/overlap, a single-use intermediate and a sufficiently small patch. The implementation targets copy-prone shared bases; an already-in-place single-use update may not benefit. Earlier CPU chains improve 2.17–2.34×; this is not an established general GPU gain.

2.5 Gather-index normalization: preserve the right batch coordinates

For source $C : [P, H, D]$ and indices $[F, N, 1]$, the last1 is an index-component axis, not another batch axis. The required read is $R[f, n, h, d] = C[\text{ids}[f, n, 0], h, d]$.

```
ids2 = reshape(ids, [F, N])      # remove ONLY the unit component
R    = gather(C, ids2)           # preserve f and n coordinates
```

This normalizes a supported depth-one gather before vectorization, avoiding the diagnosed index-projection defect. It checks the gather dimension map and matching batch/output shapes. It is a correctness repair, not head sharing; arbitrary index depth/layouts are outside this rewrite.

3 Ownership, projection and numerical transformations

3.1 Snapshot protection and borrowed-buffer copy-on-write

```
# Tensor contract: return the OLD C alongside an updated-cache result.
T = update(C, p, U)
return attention(T), C
# Required ownership if bufferized T would alias borrowed C:
T_buf = clone(C_buf); T_buf[p:p+t] = U
return attention(T_buf), C_buf
```

An in-place update through the same buffer would corrupt the returned snapshot. The early tensor pass preserves the old observation using an optimization barrier; the phase-local Stream repair follows casts to non-consuming imports and clones the update target in the supported ownership pattern. The code above illustrates that ownership requirement, not a promise to clone every update. If forwarding makes T unnecessary, a full copy can be avoided. Incorrect stock outputs are excluded from speedup baselines. Cache sharing/COW has earlier systems precedents [5, 6].

3.2 Latent projection placement: project the query instead of every key

Let $C : [N, r]$ be an already-compressed cache, $W_K : [r, d]$, and $q : [1, d]$.

```
# Before: project all N cached vectors, then compare with q.
K = C @ Wk                      # [N, d]
scores = q @ K.T
# After: one query projection and a latent-width comparison.
scores = (q @ Wk.T) @ C.T        # [1, r] @ [r, N]
```

For $N = 10000, r = 64, d = 512$, this avoids forming 10000 full-width keys and replaces their up-projections with one query projection. The value-side identity is $A(CW_V) = (AC)W_V$. **Use case:** latent-attention decode, not arbitrary standard KV caches. The pass uses shape/use and cost checks and an explicit reassociation contract: real-number equivalence does not imply bitwise FP equality. It can regress for long queries. MLA absorption is established prior work [7].

3.3 Partial-RoPE factoring: absorb content while retaining rotation

Rotary position transforms generally cannot commute through W_K. For an explicit content/rotary split, preserve the rotary contribution and transform only the content term [7, 8]:

```
# qC:512 content dims, qR:32 rotary dims; C:64 latent dims.
s = (qC @ Wk.T) @ C.T + qR @ kR.T
P = softmax(s / sqrt(512 + 32))  # preserve ORIGINAL scale
```

This equals $[q_C, q_R][CW_K, k_R]^T$. The rewritten content width 64 does not justify changing the denominator to $\sqrt{64 + 32}$. The pass matches supported split/concat graphs, retains rotary data and explicit scale, and does not claim arbitrary RoPE recognition. **Use case:** MLA-like decoupled rotary layouts.

3.4 Native-attention phase guard and decomposed fallback

Online softmax may merge *token* blocks after each score's full feature sum; it must not treat feature partial sums as extra tokens [10, 11]. In the seed-free 256-feature case, one 16-wide tile contributes scores $[\log 9, 0]$; the other 15 contribute $[0, 0]$, with values $[1, 0]$:

$$\text{correct} = \frac{9}{9+1} = 0.9, \qquad \text{wrong feature-as-token merge} = \frac{9+15}{9+1+30} = 0.6.$$

The C++ configured-IR guard protects the feature reduction under the supported CPU schedule and resource constraints; a standard-contraction fallback offers another correct route. The later audit also finds an existing upstream early-decomposition route on CPU. This is a bounded correctness safeguard, not a new attention algorithm or a proven general GPU scheduling repair.

3.5 Bounded trig argument reduction: keep large-phase RoPE accurate

```
# Conceptual repaired path, for 32 < abs(x) <= 8192:
xd = f64(x)
r  = xd - floor(xd/(2*pi) + 0.5) * (2*pi)
y  = sin_f32(f32(r))             # analogously for cos
```

Higher-precision reduction limits cancellation before the existing FP32 trig evaluation. The pass retains the already-computed FP32 phase, supports static unencoded small kernels (≤ 4096 elements) and falls back outside the guard. It is a bounded scheme, not full Payne–Hanek or an all-input accuracy proof [9]. Unmodified model validation passed 5/9 natural cases; repair enabled 9/9 under the same tolerances. Both IREE timing variants use the repair and f64-preservation flags, isolating the cache transformation.

4 Evaluation protocol

Standalone C++ MLIR 19 rewriting precedes IREE 3.13/LLVM 24 compilation (`89ece87b8985`) [12]. GPU: Tesla T4, driver 580.82.07, FP32, TF32 off. SmolLM2-135M-Instruct [15], checkpoint `12fd25f77366`: 30 layers, 9Q/3KV heads, width 64. Both IREE variants use O3, both GQA flags, identical split-argmax settings, trig repair and f64-preservation flags. Timing excludes compilation, prefill and upload: resident outputs, context synchronization, randomized order, 31 samples/repetition; median of nine repetition medians (three prompts, three repetitions). Nsight Compute 2025.3.1 app-range counters [16] use prompt 0, three independent processes and three calls/range, separately from timing. All 112 final counter exports were rechecked.

5 Winner factoring: pretrained two-candidate step

Context	Base ms	Opt. ms	Speedup	Latency lower	Total DRAM change
2048	43.264	31.815	**1.36×**	**26.46%**	−3.91%
3072	63.326	60.386	1.05×	4.64%	+1.24%
4096	82.409	77.858	1.06×	5.52%	−0.27%

This comparison isolates concat winner factoring within the repaired IREE pipeline. All natural and forced-winner cases pass, with exact cache prefixes and unchanged inputs. Worst absolute error: 5.05×10^{-5} (atol= 5×10^{-4}, rtol= 3×10^{-3}). Ordinary decode has no matches. Writes fall but reads rise: full-model DRAM reduction is inconsistent. At context 2048, kernels increase 987→1047/call and logical memcpy remains 188.8 MB/call.

6 Per-transformation operator results

Speedup = baseline time / optimized time; below 1 means slower. CPU rows are separate synthetic studies, not full-model gains. Suite figures are geometric means. Earlier T4 projection timings include output downloads; final model timings leave outputs resident. Combined gains are not additive. Full per-case times, variant names and source paths: `report/study_results/`.

Transformation	Measured effect	Workload / attribution [17]
Original GQA head reuse	1.86× CPU	Default lowering only; no extra gain established over both upstream flags plus fusion.
Conditional read forwarding	2.04× CPU	Cache A1: 187.10→91.74 μs; same best-correct baseline.
Masked gather forwarding	2.52× CPU	Same A1: 187.10→74.36 μs; alternative to conditional reads.
Winner factoring alone	1.38× CPU	2607.34→1885.37 μs; with masked forwarding: 723.42 μs, **3.60× combined**.
Adjacent-update coalescing	0.94–0.96× alone	4–16-token CPU chains. **2.17–2.34× combined** with masked forwarding; merging enables a later match.
Snapshot protection / COW	2.58× combined	109.01→42.27 μs CPU: protection *plus* masked forwarding vs fastest correct baseline; not protection alone.
Key/value projection placement	3.93× CPU; 1.20× T4	16 cases per platform vs strong unabsorbed contractions. One decode ablation: key-only 1.92×, value-only 1.95×.
Partial RoPE + projection	11.40× CPU; 1.50× T4	Four cases per platform; combined pipeline, no isolated RoPE gain. MLA absorption is prior art.
Correct contraction fallback	10.63× CPU	One wide-head case: 131.11→12.33 μs vs correct guarded fused lowering; not a gain from the guard itself.
Gather normalization / native guard	Correctness enablers	No independent valid speedup established. Wrong stock outputs are excluded from performance baselines.
Stable trig reduction	5/9→9/9 cases pass	Earlier matched-valid-case latency overhead: +0.07% (A), +0.25% (B); not a speed optimization.
Extra aggressive fusion flag	0.97× at 2048	Earlier repaired B run: 32.02→33.16 ms; does not consistently help.

7 Forwarding on T4: mechanism and negative control

Track B fixture	Base ms	Opt. ms	Speedup	DRAM MB: base → opt.
Small sparse forks	0.288	0.246	1.17×	5.831→0.128 (97.80% less)
Large sparse forks	1.777	1.230	1.44×	178.897→0.137 (99.92% less)
Dense no-rewrite control	0.545	1.268	**0.43×**	0.285→0.275 (small / variable)

Conditional forwarding, separate from the model factoring result. Track A large fixture: 1.14×, 178.228→0.132 MB in a second T4 session. Synthetic checks use independent FP64 NumPy references (atol= 5×10^{-5}, rtol= 2×10^{-3}). The unexplained no-rewrite regression and uncontrolled clocks limit causal attribution.

Limitations. One FP32 small model and GPU family; custom one-step branching, not complete serving throughput. Logical tensor elimination, physical allocation, DRAM bytes and latency are distinct claims. Incorrect ownership/numerical cases and irregular masked CUDA compile failures remain recorded. No all-GQA, mixed-precision or global-novelty claim is made.

Prior work. GQA, MLA absorption and decoupled RoPE have identifiable originating papers below. Read-over-write has classical foundations. For selection factoring, adjacent-update merging and this exact compiler repair, no definitive first paper is established here. Cache-sharing papers are related systems precedents, not attribution for our precise MLIR patterns.

References and attribution

[1] J. Ainslie, J. Lee-Thorp, M. de Jong, Y. Zemlyanskiy, F. Lebrón and S. Sanghai. *GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints*. EMNLP, 2023, pp. 4895–4901. Originating GQA reference. https://aclanthology.org/2023.emnlp-main.298/

[2] N. Shazeer. *Fast Transformer Decoding: One Write-Head is All You Need*. 2019. Introduces multi-query K/V sharing. https://arxiv.org/abs/1911.02150

[3] J. McCarthy. *Towards a Mathematical Science of Computation*. Information Processing'62, 1962, pp. 21–28. Classical functional-array foundation; not a GQA optimization paper. https://www-formal.stanford.edu/jmc/towards/towards.html

[4] C. Tinelli / SMT-LIB. *ArraysEx: Functional Arrays with Extensionality*. Formal select/store read-over-write axioms. https://smt-lib.org/theories-ArraysEx.shtml

[5] W. Kwon et al. *Efficient Memory Management for Large Language Model Serving with PagedAttention*. SOSP, 2023. KV sharing and copy-on-write precedent; not our compiler factoring pass. https://arxiv.org/abs/2309.06180

[6] S. Wang, R. Ren and L. Gui. *ForkKV: Scaling Multi-LoRA Agent Serving via Copy-on-Write Disaggregated KV Cache*. 2026. Related branching-cache system, not the first array-law reference. https://arxiv.org/abs/2604.06370

[7] DeepSeek-AI et al. *DeepSeek-V2: A Strong, Economical, and Efficient Mixture-of-Experts Language Model*. 2024. Secs.2.1.2–2.1.3 explicitly describe MLA weight absorption and decoupled RoPE. https://arxiv.org/abs/2405.04434

[8] J. Su et al. *RoFormer: Enhanced Transformer with Rotary Position Embedding*. First arXiv version, 2021. Originating RoPE reference. https://arxiv.org/abs/2104.09864

[9] M. H. Payne and R. N. Hanek. *Radian Reduction for Trigonometric Functions*. ACM SIGNUM Newsletter 18(1), 1983, pp. 19–24. Seminal argument-reduction reference; our bounded f64 scheme is not its full algorithm. https://doi.org/10.1145/1057600.1057602

[10] M. Milakov and N. Gimelshein. *Online Normalizer Calculation for Softmax*. 2018. Online normalization background. https://arxiv.org/abs/1805.02867

[11] T. Dao, D. Y. Fu, S. Ermon, A. Rudra and C. Ré. *FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness*. NeurIPS, 2022. Fused IO-aware attention background, not implemented anew in this project. https://arxiv.org/abs/2205.14135

[12] C. Lattner et al. *MLIR: A Compiler Infrastructure for the End of Moore's Law*. 2020. https://arxiv.org/abs/2002.11054

[13] IREE contributors. *GeneralizeLinalgNamedOps; BubbleUpExpandShapes; compilation pipelines*. Source pinned to compiler commit `89ece87b89852100c0a4ac3d515e14842d090967`. Implementation evidence for existing transformations, not a GQA-specific paper. Pinned source: GeneralizeLinalgNamedOps.cpp and BubbleUpExpandShapes.cpp.

[14] IREE contributors. *MLIR pass reference: GlobalOptimization and DispatchCreation*. Named-op generalization, reshape propagation through reductions and elementwise fusion. The observed behavior is pinned by the source/IR evidence, since online documentation can change. https://iree.dev/reference/mlir-passes/GlobalOptimization/ https://iree.dev/reference/mlir-passes/DispatchCreation/

[15] Hugging Face. *SmolLM2-135M-Instruct model card and checkpoint*. Pinned revision in the evaluation protocol. https://huggingface.co/HuggingFaceTB/SmolLM2-135M-Instruct

[16] NVIDIA. *Nsight Compute Profiling Guide*. Application-range replay, metric coverage and cache/clock controls. https://docs.nvidia.com/nsight-compute/ProfilingGuide/

[17] CS6886 project. *Attention and KV-cache compiler optimizations: implementation, raw results and reproduction*. Experiment-source snapshot `71cc40ae53c5e60f2a690e67ffbc64e85969e64f`. https://github.com/lazarsalt1441/cs6886_project